# Tokenisation over Bounded Alphabets is Hard

*Violeta Kastreva*, Philip Whittington*, Dennis Komm, Tiago Pimentel*
**equal contribution*

ETH zürich

**We show that tokenisation over binary alphabets is APX-hard, and even over unary alphabets it is NP-complete (for direct tokenisation).** This means unary tokenisation cannot be solved efficiently, and binary tokenisation cannot be approximated efficiently.

## Why do we tokenise?

- Good tokenisers can increase model performance and efficiency, bad tokenisers can lead to problems
- How to choose a tokeniser?
  ⇒ Main objective: model performance
  → Not practically measurable
  ⇒ Proxy objectives: compression, unigram log-probability, ...
- We focus on **compression**

## Bottom-up Tokenisation

- Incrementally merge tokens to get tokenised string
- Example:

Merges: $[h \circ e,\ r \circ e,\ t \circ he]$

Input: they are there

| Pretok.: | they | _are | _there |
|---|---|---|---|
| Chars: | $\langle t,\ h,\ e,\ y\rangle$ | $\langle _,\ a,\ r,\ e\rangle$ | $\langle _,\ t,\ h,\ e,\ r,\ e\rangle$ |
| $h + e$ | $\langle t,\ he,\ y\rangle$ | $\langle _,\ a,\ r,\ e\rangle$ | $\langle _,\ t,\ he,\ r,\ e\rangle$ |
| $t + he$ | $\langle t,\ he,\ y\rangle$ | $\langle _,\ a,\ re\rangle$ | $\langle _,\ t,\ he,\ re\rangle$ |
| $r + e$ | $\langle the,\ y\rangle$ | $\langle _,\ a,\ re\rangle$ | $\langle _,\ the,\ re\rangle$ |

Output: $\{a,\ e,\ h,\ r,\ t,\ y,\ he,\ re,\ t(he)\}$

- How to select the merge sequence?
  ⇒ BPE finds these merges greedily: given a dataset, it selects the most frequent merge at a time.

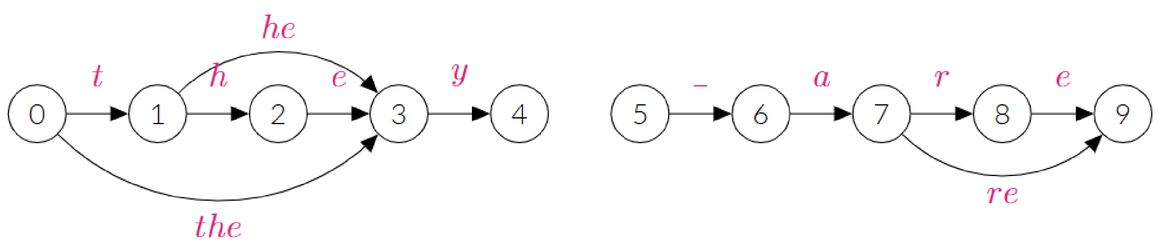
## Direct Tokenisation

- Choose tokens which result in shortest possible tokenised string
- Example:

$\{a,\ e,\ h,\ r,\ t,\ y,\ he,\ _,\ re,\ the\}$

Input: they are there

Pretok.: they _are _there




Output: $\langle the,\ y\rangle$ $\langle _,\ a,\ re\rangle$ $\langle _,\ the,\ re\rangle$

- How to select the set of tokens?
  ⇒ Unclear, but could use BPE or UnigramLM.

## Tokenisation over n-ary alphabets

- Prior work (Kozma & Voderholzer, Whittington et al., Lim et al.) proved tokenisation is NP-complete for unbounded alphabets
- Practical tokenisers use fixed alphabets (e.g. bytes / ASCII / Unicode)
- We therefore study tokenisation over bounded n-ary alphabets

## What we prove

- For every alphabet of size **n ≥ 2,** both bottom-up and direct tokenisation are **NP-complete**
- Already over a binary alphabet, tokenisation is **APX-hard**, so even approximation is hard (unless P=NP)
- We further show that direct tokenisation is **NP-complete** even over a unary alphabet

## Reduction Ideas

**`max-2-SAT`**

Input: variables $\mathcal{X} = \{X_j\}_{j=1}^{J}$, clauses $\mathcal{C} = \{(L_i^1 \vee L_i^2)\}_{i=1}^{I}$, target $\gamma$

Question: Can $\gamma$ clauses be satisfied?

Task: assign $\mathtt{T}$ or $\mathtt{F}$ to $X_j$

*Example input:* $(\mathbf{x}_1 \vee \mathbf{x}_2) \wedge (\neg\mathbf{x}_1 \vee \mathbf{x}_2) \wedge (\mathbf{x}_1 \vee \neg\mathbf{x}_2) \wedge (\neg\mathbf{x}_1 \vee \neg\mathbf{x}_2)$

### Direct Binary Tokenisation

Encode $\mathbf{x}_j^{\mathtt{T}} = 0^{2j-1}$ and $\mathbf{x}_j^{\mathtt{F}} = 0^{2j}$

Input: alphabet $\Sigma = \{0,\ 1\}$, dataset $\mathcal{D} = \{1L_i^1 1 L_i^2 1\}_{i=1}^{I} \cup \mathcal{D}'$

vocabsize $K := 5J$, target $\delta := f(I, J) - \gamma$

Objective: can $\mathcal{D}$ be compressed to $\delta$ symbols?
(with at most $K$ tokens)

Task: choose between tokens $1\mathbf{x}_j^{\mathtt{T}}1$ or $1\mathbf{x}_j^{\mathtt{F}}1$ for each variable

*Example output:* $\{1\mathbf{x}_1^{\mathtt{T}}1\mathbf{x}_2^{\mathtt{T}}1,\ 1\mathbf{x}_1^{\mathtt{F}}1\mathbf{x}_2^{\mathtt{T}}1,\ 1\mathbf{x}_1^{\mathtt{T}}1\mathbf{x}_2^{\mathtt{F}}1,\ 1\mathbf{x}_1^{\mathtt{F}}1\mathbf{x}_2^{\mathtt{F}}1\}$

### Direct Unary Tokenisation

With only one symbol, strings differ only by their lengths

Reduction from **`vertex-cover`**: can we select at most $\psi$ vertices covering all edges?

Input: $\underbrace{\mathrm{enc}(v_j)}_{\text{vertex-string}}$, $\underbrace{\mathrm{enc}(v_j) + B}_{\text{cover-string}}$, $\underbrace{\mathrm{enc}(u) + \mathrm{enc}(v) + B}_{\text{edge-string}}$

vocabsize $K := \psi$, target $\delta := f(I, J) - \psi$

Task: choose token $\mathrm{enc}(v_j) + B$ to include $v_j$ in the cover

## Implications and Future Work

- The **hardness of tokenisation is not an artifact of large alphabets**, but inherent to the problem itself
  ⇒ Even over binary and, in part, unary alphabets, exact **optimisation remains hard**
  ⇒ For binary alphabets, APX-hardness rules out arbitrarily good polynomial-time approximation unless P=NP.
- Open questions:
  ⇒ Does binary tokenisation admit a constant-factor approx.?
  ⇒ What is the best compression achievable by efficient tokenisation algorithms?
  ⇒ Do properties of natural language make practical instances easier?
  ⇒ What is the hardness of other objectives and variants?